Université de Sorbonne Paris Nord **Algorithmique du texte** **Année 2022-2023**
Institut Galilée **Contrôle continu**

Toutes les réponses doivent être soigneusement justifiées.
La qualité de la rédaction interviendra de manière importante dans la notation.

Dans tout l'énoncé, pour un mot `x`, `|x|` désigne la taille de `x`, ou encore le nombre de ses lettres.

Q1. On considère l'algorithme du Boyer–Moore de recherche d'un motif m dans un texte t, lorsque pour choisir le bon déplacement de la fenêtre glissante, c'est la règle "bon suffixe" qui est utilisée.

(a) Donner un algorithme qui a partir du motif calcule la table des bons déplacements à effectuer lorsque l'on applique la règle du bon suffixe.

(b) Quelle est la complexité votre algorithme ?

(c) Quelle est la complexité totale de la recherche d'un motif par l'algorithme de Boyer Moore avec la règle du bon suffixe ?

Q2. Etant donné un mot `u = u[0 .. m−1]`, sa table des bords `Bord` est définie par `Bord[0]=−1`, et `Bord [j]=` la longueur maximale d'un bord (propre) de `u[0 .. j−1]` pour $0 < j \leq m$.

(a) Donner la table des bords du mot `abaabaaabaa`.

Soit `x` et `y` deux mots sur l'alphabet {`a`,`b`}. Soit `B` la table des bords du mot `xcy` où `c` est une lettre différente de `a` et de `b`.

(b) Supposons qu'il existe une position `i` telle que `B[i]=|x|`. Donner en justifiant le facteur de `xcy` qui commence à la position `i-|x|+1`. En déduire que `x` est alors un facteur de `y`.

(c) Supposons que `x` est un facteur de `y` à la position $\widehat{j}$ dans `y`. Donner en justifiant la valeur de `B[j+|x|-1]`.

(d) En déduire une stratégie pour trouver toutes les occurrences de `x` et de `y`, en utilisant la table des bords de `xcy` où c est une lettre différente de `a` et de `b`.

(e) En pratique, pour trouver toutes les occurrences de `x` et de `y`, on commence par calculer la table des bords de `x` et non de `xcy`. Pourquoi ?

(f) Donner le pseudo-code d'un algorithme qui calcule toutes les occurrences de `x` dans `y` à partir de la table des bords de `x`. Donner en justifiant le temps d'exécution de cet algorithme.

(g) Un mot `w` est appelé une couverture d'un mot `t`, si `w` est préfixe de `t` et si deux positions consécutives `i` et `j, (i<j)` de `w` dans `t` vérifient `j−i <= |w|`. Notez que deux occurrences consécutives de `w` peuvent se chevaucher. Par exemple, `aba` est une couverture de `ababaabab`. C'est la plus courte couverture de `ababaabab`. Etant donné deux mots `w` et `t`, donner en justifiant un algorithme qui fonctionne en temps $O(|t|)$ et qui répond si `w` est une couverture de `t`.

Q3. On considère l'algorithme suivant :

```
tableau d'entiers  Algo1(string x)
1: m= |x| ; k = 0
2: KMPnext[0] = -1 ; j = -1
3: for i = 0 to m-1 do
4:       if x[i] == x[k] then
5:                KMPnext[i] = KMPnext[k]
6:       else
7:                KMPnext[i] = k
8:                do k = KMPnext[k]
9:                while k >= 0 and x[i] != x[k]
10:      end if
11:      k = k + 1
12: end for
13: KMPnext[m] = k
14: return KMPnext
```

(a) Remplissez le tableau ci-dessous, en exécutant Algo1 pour chacun des mots `ababa`, `abcacabb` et en inscrivant dans chaque ligne le résultat retourné par l'algorithme.

strings	0	1	2	3	4	5	6	7	8	9	10
ababa											
abcacabb											

(*b*) Quelle est la complexité de Algo1 ?
(*c*) Quel est le tableau calculé par Algo1 ?
(*d*) Donner un algorithme Algo2 qui prend en entrée un mot `t` ainsi que le tableau retourné par Ago1 avec un mot `w` comme entrée, et calcule les positions des occurrences de `w` dans `t`.

Q4. On considère l'alphabet `A={a,b,c}` et `X` un ensemble fini de mots construits sur `A`. L'automate de Aho-Corazik associé à l'ensemble X est noté $AC(X)$.
(*a*) Donner le trie construit avec les mots de l'ensemble `X= aabbaba,aa,ab,bba`.
(*b*) Donner en justifiant les valeurs de la fonction de suppléance et de la fonction de sortie associées à chaque noeud du trie.
(*c*) Dessiner l'automate de Aho-Corasick associé à `X`.
(*d*) Donner un algorithme qui prend en entrée un automate de Aho-Corasick d'un ensemble `X` de motifs et un texte `t` de longueur n et qui recherche les motifs `X` dans le texte `t`. En donner la complexité.
(*e*) Donner la trace de l'exécution de votre algorithme pour calculer les positions des mots de `X` dans le texte `t = aabbabaabbababababaabb`.

Q5.
(*a*) Écrire un algorithme naïf qui à partir d'un texte construit l'arbre des suffixe du texte.
(*b*) Donner la complexité de votre algorithme.

Q6. On considère le mot `z=aabbabb$`.
(*a*) Donner le trie construit avec tous les suffixes de `z`. Quel est l'intérêt d'avoir ajouté un caractère de contrôle à la fin de `z` ?
(*b*) Donner un exemple de mot de longueur n sur l'alphabet `{a,b}` dont le trie des suffixes est de taille $\Omega(n^2)$ (de taille asymptotiquement et à une constante multiplicative près, plus grand que n^2).
(*d*) Quels sont la taille et le temps pour calculer l'arbre des suffixes d'un texte `t` de longueur n ?
(*e*) Donner un algorithme qui prend en entrée un motif `w` de longueur m et l'arbre des suffixes d'un texte `t` de longueur n et qui recherche les occurrences du motif `w` dans le texte `t`. En donner la complexité.